PROJECT 1: Web Application Vulnerability Scanner

Code:

```
import requests
from bs4 import BeautifulSoup
import re

# Common payloads
SQL_PAYLOAD = "' OR '1'='1"
XSS_PAYLOAD = "<script>alert('XSS')</script>"

def scan_sql_injection(url):  1 usage
    print("\n[+] Testing SQL Injection...")
    test_url = url + "?id=" + SQL_PAYLOAD
    response = requests.get(test_url)

    errors = ["sql", "mysql", "syntax", "error"]

    for error in errors:
        if re.search(error, response.text, re.IGNORECASE):
            print("[!] Possible SQL Injection vulnerability detected!")
            return True

    print("[-] No SQL Injection vulnerability detected.")
    return False
```

```python
def scan_xss(url):  1 usage
    print("\n[+] Testing XSS...")
    test_url = url + "?q=" + XSS_PAYLOAD
    response = requests.get(test_url)

    if XSS_PAYLOAD in response.text:
        print("[!] Possible XSS vulnerability detected!")
        return True

    print("[-] No XSS vulnerability detected.")
    return False


def main():  1 usage
    url = input("Enter target URL (example: http://testphp.vulnweb.com): ")

    scan_sql_injection(url)
    scan_xss(url)

    print("\nScan Completed.")


if __name__ == "__main__":
```

```python
def scan_xss(url):  1 usage
        return True

    print("[-] No XSS vulnerability detected.")
    return False


def main():  1 usage
    url = input("Enter target URL (example: http://testphp.vulnweb.com): ")

    scan_sql_injection(url)
    scan_xss(url)

    print("\nScan Completed.")


if __name__ == "__main__":
    main()
```

Output:

```
"C:\Users\MUMTAZ SHAIKH\PyCharmMiscProject\.venv\Scripts\python.exe" "C:\Users\MUMTAZ SHAIKH\PyCharmMiscProject\project.py"
Enter target URL (example: http://testphp.vulnweb.com): http://testphp.vulnweb.com

[+] Testing SQL Injection...
[!] Possible SQL Injection vulnerability detected!

[+] Testing XSS...
[-] No XSS vulnerability detected.

Scan Completed.

Process finished with exit code 0
```

Explanation :

Web Application Vulnerability Scanner

1️⃣ Introduction

In today's digital world, web applications are widely used for online services such as banking, shopping, and education. However, many web applications contain security vulnerabilities that attackers can exploit. Common vulnerabilities include SQL Injection (SQLi) and Cross-Site Scripting (XSS). These vulnerabilities can lead to data theft, unauthorized access, and system compromise.

A Web Application Vulnerability Scanner is a tool designed to automatically detect security flaws in web applications. It helps developers and security professionals identify and fix vulnerabilities before attackers can exploit them.

---

2 Abstract

This project focuses on developing a Python-based Web Application Vulnerability Scanner that detects common web vulnerabilities such as SQL Injection and Cross-Site Scripting (XSS). The scanner sends specially crafted payloads to target URLs and analyzes server responses to identify potential security weaknesses.

The tool uses HTTP requests to interact with web applications and applies pattern matching techniques to detect error messages or reflected scripts in responses. The scanner provides output indicating whether a vulnerability is detected. This project demonstrates the importance of automated security testing in protecting web applications.

---

3 Tools Used

- **Python (Programming Language)**
- **requests library (for sending HTTP requests)**
- **BeautifulSoup (for parsing HTML content)**
- **re (Regular Expressions for pattern matching)**
- **PyCharm IDE (for development and testing)**
- **Test Website: http://testphp.vulnweb.com**

---

4 Steps Involved in Building the Project

1. **Requirement Analysis**
   Identified common vulnerabilities such as SQL Injection and XSS to be detected.
2. **Payload Creation**
   Created test payloads such as:
   - **SQL Injection: ' OR '1'='1**
   - **XSS: <script>alert('XSS')</script>**
3. **Sending HTTP Requests**
   Used the requests library to send payloads to target URLs.
4. **Response Analysis**
   Checked the server response for:
   - **SQL error messages (e.g., syntax error, MySQL error)**
   - **Reflected script tags for XSS detection**
5. **Result Display**
   Displayed vulnerability detection results in the terminal.
6. **Testing**
   Tested the scanner on a deliberately vulnerable test website.

5️⃣ Working Principle

The scanner works by injecting malicious input into URL parameters and observing the server's response. If the

application returns database errors or reflects injected scripts, the scanner flags the issue as a possible vulnerability. This method is based on basic black-box testing principles.

6️⃣ Conclusion

The Web Application Vulnerability Scanner successfully demonstrates how automated tools can identify common web vulnerabilities such as SQL Injection and XSS. This project highlights the importance of secure coding practices and regular security testing. Although the scanner performs basic detection, it provides a foundation for building more advanced security testing tools.

Future improvements may include:

- **Form-based vulnerability scanning**
- **Severity classification**
- **Report generation in PDF format**
- **Integration with OWASP Top 10 checks**